Manual del Programador — Piedra, Papel o Tijera v1.0.0

LPR 2026 · Actividad 7 · E.E.S.T. N° 100 "Toy Story"

1. Objetivo técnico

Demostrar la diferencia entre memoria RAM (volátil) y almacenamiento secundario (no volátil) mediante un juego que registra cada partida en una `struct` y la persiste en un archivo de texto con la librería `<fstream>`.

2. Entorno y compilación

- Lenguaje: C++11.
- Compilador: `g++` (MinGW / MSYS2 en Windows).
- Editor: Visual Studio Code, terminal PowerShell.

```
g++ -std=c++11 src/main.cpp -o src/piedrapapelotijera.exe
.\src\piedrapapelotijera.exe
```

Ejecutar desde la **raíz del repositorio**: las rutas de archivo son relativas a esa carpeta.

3. Estructura del proyecto

Ruta	Contenido
`src/main.cpp`	Código fuente completo
`src/historial_partidas.txt`	Archivo de persistencia (se genera al jugar)
`docs/`	Informe, manuales y CHANGELOG
`capturas/`	Evidencias de ejecución (.png)

4. Librerías utilizadas

Librería	Uso
`<iostream>`	Entrada/salida por consola (`cin`, `cout`, `cerr`)
`<fstream>`	Flujos de archivo (`ofstream`, `ifstream`)
`<string>`	Tipo `string` y `getline`
`<cstdlib>`	`rand()` y `srand()`
`<ctime>`	`time(0)` para la semilla aleatoria

5. Modelo de datos

```
struct Partida {
  int idPartida;
  string jugador;
  string eleccionJugador;
  string eleccionIA;
  string resultado;
};
```

Agrupa cinco datos heterogéneos en una sola entidad, de modo que las funciones reciben un único parámetro (`const Partida&`) en lugar de cinco variables sueltas.

6. Funciones

Función	Descripción
`main()`	Inicializa la semilla, muestra el menú en un `do...while` y deriva a cada opción
`jugarPartida(int& contadorID)`	Pide nombre y jugada, genera la jugada de la IA, evalúa el resultado y llama a `guardarEnDisco`
`guardarEnDisco(const Partida& p)`	Abre el archivo con `ios::app` y agrega una línea CSV
`leerHistorialDisco()`	Abre el archivo con `ifstream`, lee con `getline` delimitado por comas y muestra los registros

7. Flujo de ejecución

1. `main()` ejecuta `srand(time(0))` y entra al menú.
2. **Opción 1:** `jugarPartida` crea una `Partida`, completa sus campos y llama a `guardarEnDisco`.
3. **Opción 2:** `leerHistorialDisco` recupera y muestra todos los registros.
4. **Opción 3:** termina el bucle y el programa finaliza.

8. Persistencia en disco

Escritura

```
ofstream archivo("src/historial_partidas.txt", ios::app);
```

El modo `ios::app` (append) agrega al final sin borrar el contenido existente. Sin esa bandera, cada apertura truncaría el archivo y solo quedaría la última partida.

Formato del archivo (CSV)

```
idPartida,jugador,eleccionJugador,eleccionIA,resultado
```

Ejemplo:

```
1,Ana,Piedra,Papel,Derrota
2,Ana,Tijera,Papel,Victoria
```

Lectura

```
ifstream archivo("src/historial_partidas.txt");
while (getline(archivo, idStr, ',') && ... && getline(archivo, resultado)) { ... }
```

Los primeros cuatro campos se leen con delimitador `,` y el último hasta el fin de línea.

9. Manejo del búfer de teclado

Tras `cin >> opcion`, el Enter queda en el búfer. Si se llamara a `getline` directamente, leería una línea vacía. Por eso `jugarPartida` ejecuta `cin.ignore()` antes de `getline(cin, p.jugador)`.

10. Limitaciones conocidas

- **IDs repetidos entre sesiones:** `contadorID` arranca en 1 en cada ejecución, por lo que el

historial puede tener varios registros con el mismo ID. Mejora posible: leer el último ID del archivo al iniciar.

- **Sin validación de jugada:** cualquier valor distinto de 1 o 2 se interpreta como Tijera.
- **Sin validación de entrada:** una letra en lugar de un número deja a `cin` en estado de error.
- **Nombres con comas:** rompen el formato CSV al leer el historial.
- **Ruta relativa:** solo funciona si se ejecuta desde la raíz del repositorio.

11. Adaptación al Proyecto Integrador Anual

Para registrar eventos reales del prototipo (lecturas de sensores, logs, stock, fichajes), basta con reemplazar los campos de `Partida` y mantener la misma técnica de `ofstream` con `ios::app` e `ifstream`.

12. Control de versiones

Repositorio con doble remoto (GitHub + GitLab). Ver `docs/CHANGELOG.md` para el historial de cambios.